

BlackLine

Tattoo & Piercing Marketplace

Documentación Técnica de Algoritmos Core

Versión: 2.0.0
Fecha: Mayo 2026
Stack: Django REST + Angular 19 + PostgreSQL
Rama: BL-95-RF-4 (Contraoferta de precio/fecha)

Este documento describe los algoritmos, flujos de datos y decisiones arquitectónicas centrales del sistema BlackLine.

Contents

1	Visión General de la Arquitectura	3
2	Algoritmo de Precio Canónico	3
2.1	Descripción	3
2.2	Fórmula	3
2.3	Multiplicadores por zona corporal	3
2.4	Diagrama de flujo	5
3	Flujo Cotizador → Match → Cita	6
3.1	Descripción	6
3.2	Gestión de estado en cliente	6
3.3	Endpoints involucrados	6
3.4	Diagrama de secuencia	7
4	Máquina de Estados de la Cita	7
4.1	Descripción	7
4.2	Transiciones válidas	7
4.3	Diagrama de estados	8
5	Gestión de Sesión JWT	8
5.1	Descripción	8
5.2	Reglas de seguridad	8
5.3	Flujo de renovación automática	8
5.4	Diagrama de secuencia	9
6	Esquema de Base de Datos (ERD)	9
6.1	Descripción	9
6.2	Relaciones principales	10
6.3	Diagrama entidad-relación	11
7	Normalización de Ciudades	12
7.1	Descripción	12
7.2	Problema que resuelve	12
7.3	Invariantes del sistema	12
7.4	Diagrama de flujo	13
8	Flujo de Contraoferta de Precio y Fecha (BL-95-RF-4)	14
8.1	Descripción	14
8.2	Validaciones del serializer	14
8.3	Diagrama de secuencia	15
9	Validación de Solapamiento en CalendarBlock	15
9.1	Descripción	15
9.2	Algoritmo de detección	15
9.3	Casos borde	16
9.4	Diagrama de decisión	17

10 Pipeline OpenAPI → Tipos Angular	18
10.1 Descripción	18
10.2 Cuándo regenerar	18
10.3 Comandos del pipeline	18
10.4 Diagrama de dependencias	18
11 Flujo de Cuestionario de Salud (HealthConsent)	18
11.1 Descripción	18
11.2 Campos del modelo	19
11.3 Diagrama de secuencia	20
12 Algoritmo de Match (Filtrado de Artistas)	20
12.1 Descripción	20
12.2 Parámetros de entrada	21
12.3 Pipeline de la query (7 pasos)	21
12.4 Diagrama de flujo	22
13 Guards y Permisos por Rol (Angular)	23
13.1 Descripción	23
13.2 Guards implementados	23
13.3 Árbol de rutas protegidas	23
13.4 Diagrama de flujo	24
14 Permisos DRF en el Backend	24
14.1 Descripción	24
14.2 Tabla de permisos por endpoint	25
14.3 Diagrama de permisos	25
15 Cálculo del Rango de Precio en el Frontend	25
15.1 Descripción	25
15.2 Fórmula frontend	26
15.3 Diferencia entre precio frontend y precio backend	26
15.4 Diagrama de flujo	27
16 Índices de Base de Datos (Performance)	28
16.1 Descripción	28
16.2 Índices recomendados	28
16.3 Implementación con migraciones Django	28
16.4 Diagrama de índices	28
17 Endpoint GET /api/quotes/cities/ (Backlog UX)	28
17.1 Descripción	29
17.2 Implementación propuesta	29
17.3 Integración con el autocomplete Angular	29
17.4 Diagrama de flujo	30
18 Resumen de Endpoints Críticos	31

1. Visión General de la Arquitectura

BlackLine sigue una arquitectura **SPA + REST API** con separación estricta de responsabilidades entre frontend y backend.

Frontend (Angular 19)

- Componentes por feature module
- Signals + NgRx para estado reactivo
- Guards e interceptores HTTP
- Tipos generados desde OpenAPI

Backend (Django REST)

- ViewSets + Serializers (DRF)
- Servicios de lógica de negocio
- Modelos ORM con PostgreSQL
- JWT con SimpleJWT

Toda respuesta sigue el contrato:

- **Éxito:** { data: T, message?: string }
- **Error:** { data: null, errors: Record<string, string[]> }
- **Lista paginada:** { count, next, previous, results: T[] }

2. Algoritmo de Precio Canónico

2.1. Descripción

El cálculo de precio es la operación económica central del sistema. El backend es la **única fuente de verdad**; el frontend muestra únicamente un rango estimado.

2.2. Fórmula

Fórmula canónica (backend/apps/quotes/views.py)

```
estimated_price = MAX(
    size_cm × base_hourly_rate × zone_multiplier × color_multiplier,
    minimum_setup_fee
)
```

El rango que se muestra al cliente se calcula en el frontend:

$$\text{price_min} = \text{estimated_price}, \quad \text{price_max} = \text{estimated_price} \times 1.35$$

usando `BASE_RATE_MXN = 150` como tarifa de mercado promedio.

2.3. Multiplicadores por zona corporal

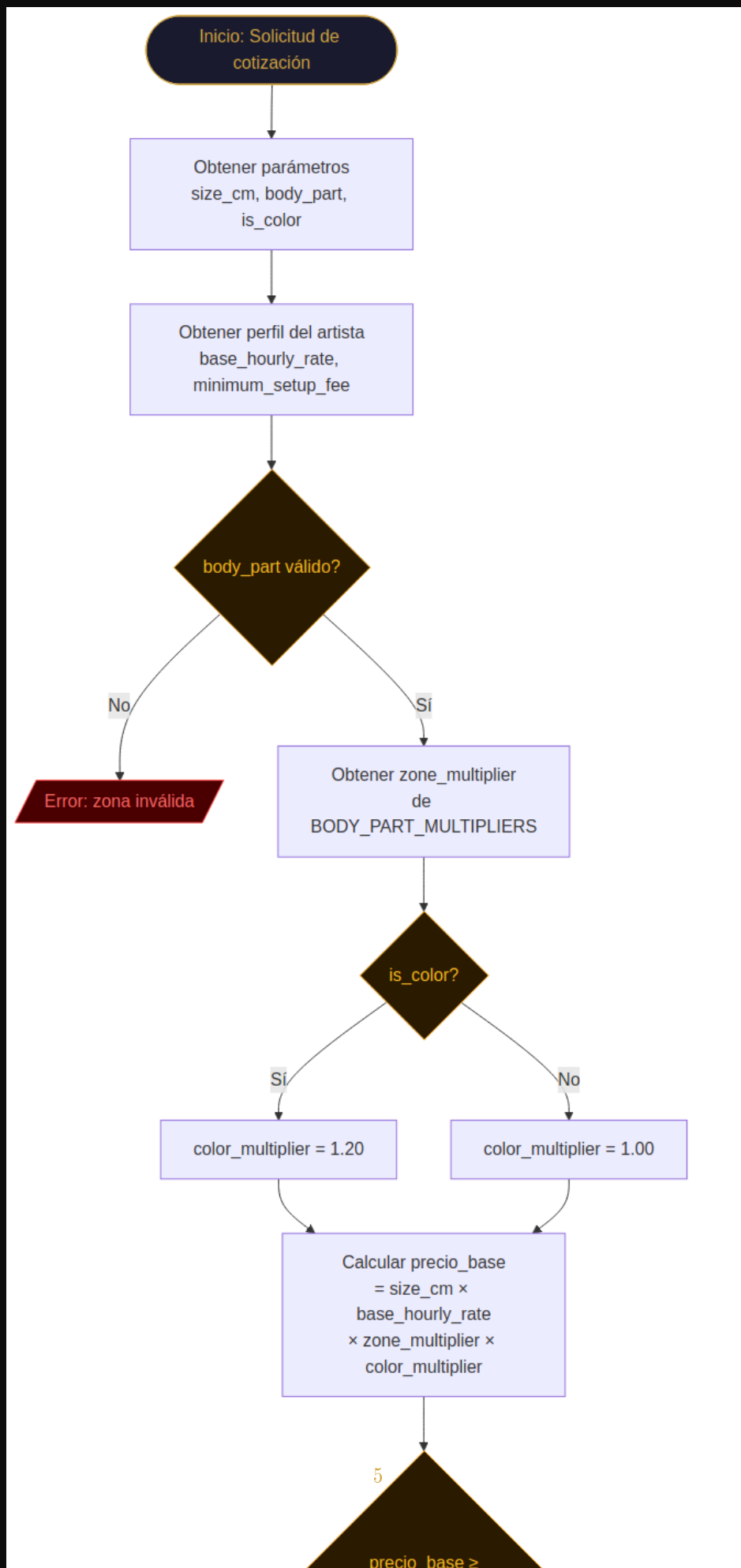
Zona	Factor	Zona	Factor
BRAZO	1.00	ESPALDA	1.30
ANTEBRAZO	1.00	PECHO	1.25
PIERNA	1.10	COSTILLAS	1.50
HOMBRO	1.05	CUELLO	1.40
MANO	1.35	PIE	1.35

Multiplicador de color: $\times 1.20$ si `is_color = true`, $\times 1.00$ si blanco y negro.

Regla de sincronización

Los valores de `BODY_PART_MULTIPLIERS` en `backend/apps/quotes/views.py` son la fuente de verdad. Si se modifican, deben actualizarse también en `cotizador.component.ts`.

2.4. Diagrama de flujo



3. Flujo Cotizador → Match → Cita

3.1. Descripción

El flujo principal del cliente recorre tres etapas: creación de la solicitud de cotización, búsqueda de artistas compatibles (match), y agendamiento de la cita.

3.2. Gestión de estado en cliente

QuoteService.lastQuote (Angular Signal)

```
ngOnInit():
1. Si lastQuote() → usa in-memory (restaurado desde sessionStorage)
2. Si null → getMyQuotes() y carga la más reciente de la API
setLastQuote(quote):
Escribe Signal + sessionStorage atómicamente
```

3.3. Endpoints involucrados

Método	Endpoint	Acción
POST	/api/quotes/	Crea QuoteRequest
GET	/api/quotes/match/	Artistas filtrados por ciudad + estilo
POST	/api/quotes/appointments/	Crea Appointment (PENDING)
POST	/api/quotes/appointments/{id}/health-consent/	Guarda Health-Consent

3.4. Diagrama de secuencia

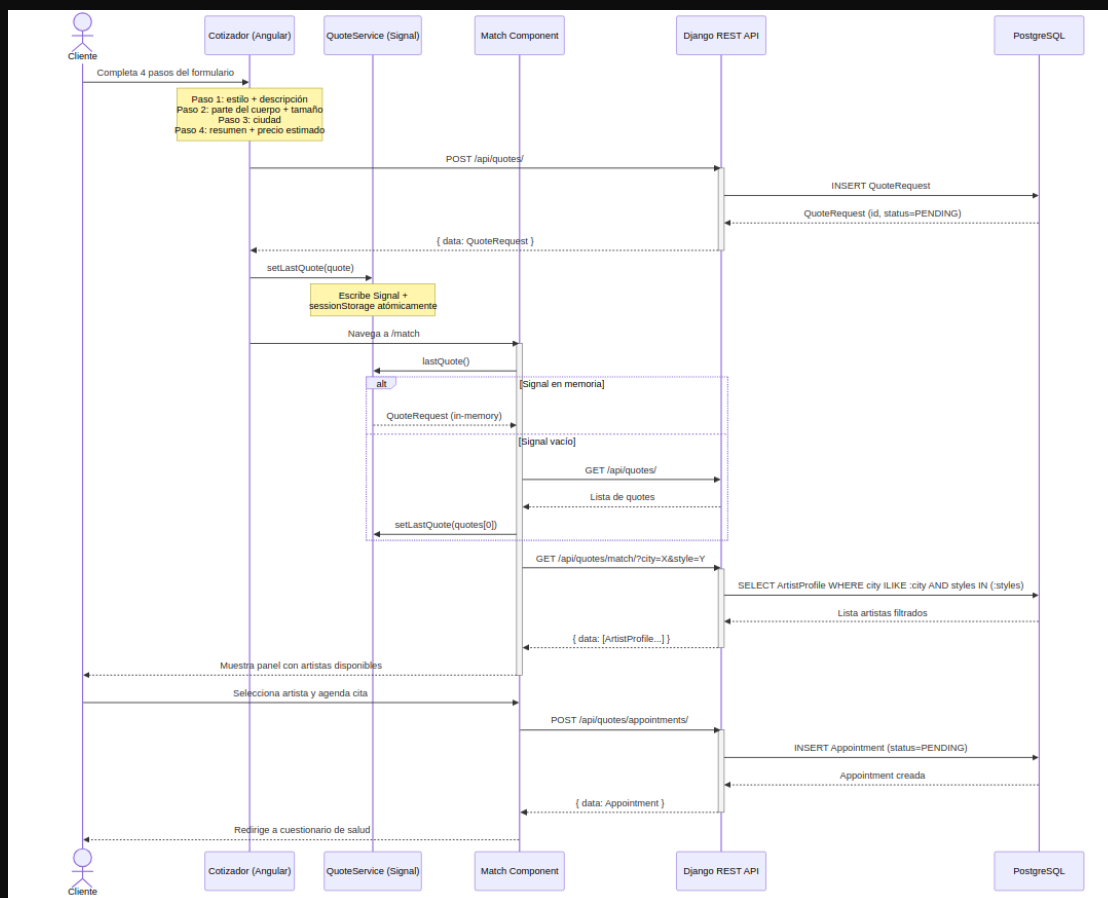


Figure 2: Secuencia completa: creación de cotización, búsqueda de artista y agendamiento de cita

4. Máquina de Estados de la Cita

4.1. Descripción

Las citas (Appointment) siguen una máquina de estados estricta. Solo el artista puede iniciar transiciones desde PENDING.

4.2. Transiciones válidas

Estado origen	Estado destino	Condición
PENDING	APPROVED	Artista acepta cita
PENDING	REJECTED	Artista rechaza cita
PENDING	COUNTER_OFFER	Artista propone cambios
COUNTER_OFFER	APPROVED	Cliente acepta contraoferta
COUNTER_OFFER	REJECTED	Cliente rechaza contraoferta

PATCH /api/quotes/appointments/{id}/status/

Body para contraoferta: { "status": "COUNTER_OFFER", "price": 1500, "scheduled_at": "..."}

4.3. Diagrama de estados

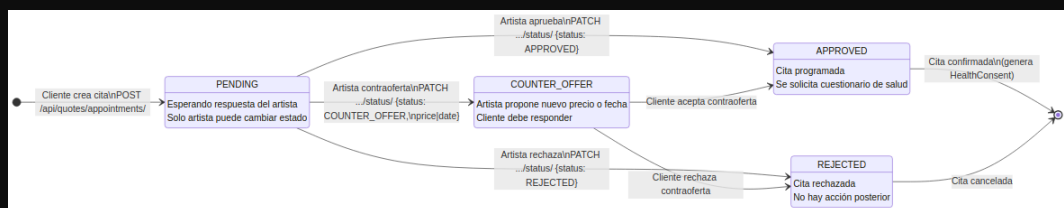


Figure 3: Máquina de estados de Appointment: transiciones y actores responsables

5. Gestión de Sesión JWT

5.1. Descripción

El sistema usa JWT con rotación de refresh tokens. El interceptor HTTP gestiona automáticamente la renovación transparente para el usuario.

5.2. Reglas de seguridad

Política de almacenamiento (OWASP)

- **PERMITIDO:** `sessionStorage` — se limpia al cerrar el navegador
- **PROHIBIDO:** `localStorage` — persiste entre sesiones, riesgo XSS
- Los tokens se eliminan completamente en `logout()`
- El refresh token se invalida en el backend (blacklist)

5.3. Flujo de renovación automática

Cuando el servidor retorna 401 `Unauthorized`, el `JwtInterceptor` realiza automáticamente:

1. Recupera el refresh token de `sessionStorage`
2. Llama a `POST /api/auth/token/refresh/`
3. Almacena el nuevo access token
4. Reintenta la petición original con el token renovado
5. Si el refresh también falla, redirige al login

5.4. Diagrama de secuencia

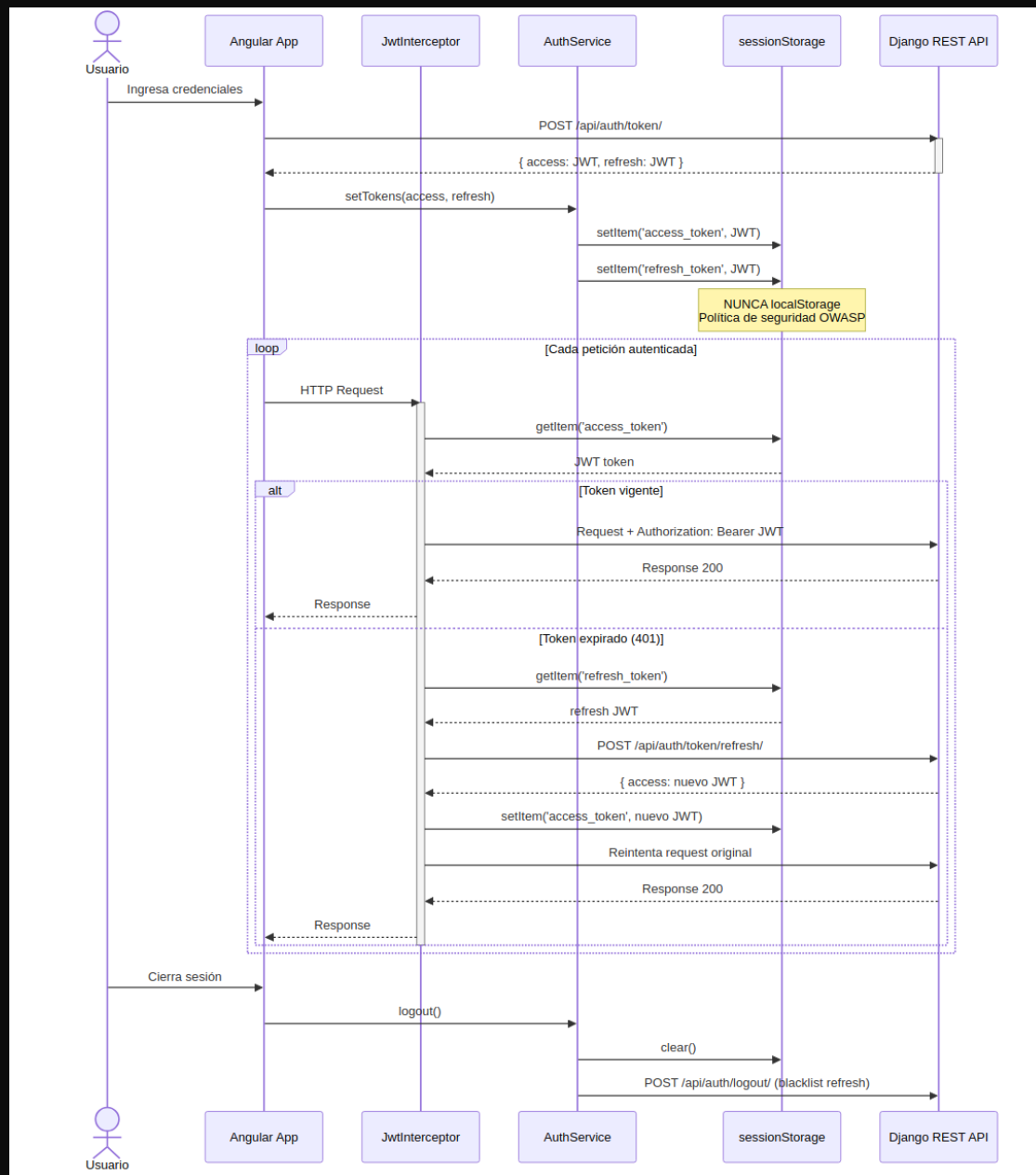


Figure 4: Ciclo de vida del JWT: autenticación, renovación automática y cierre de sesión

6. Esquema de Base de Datos (ERD)

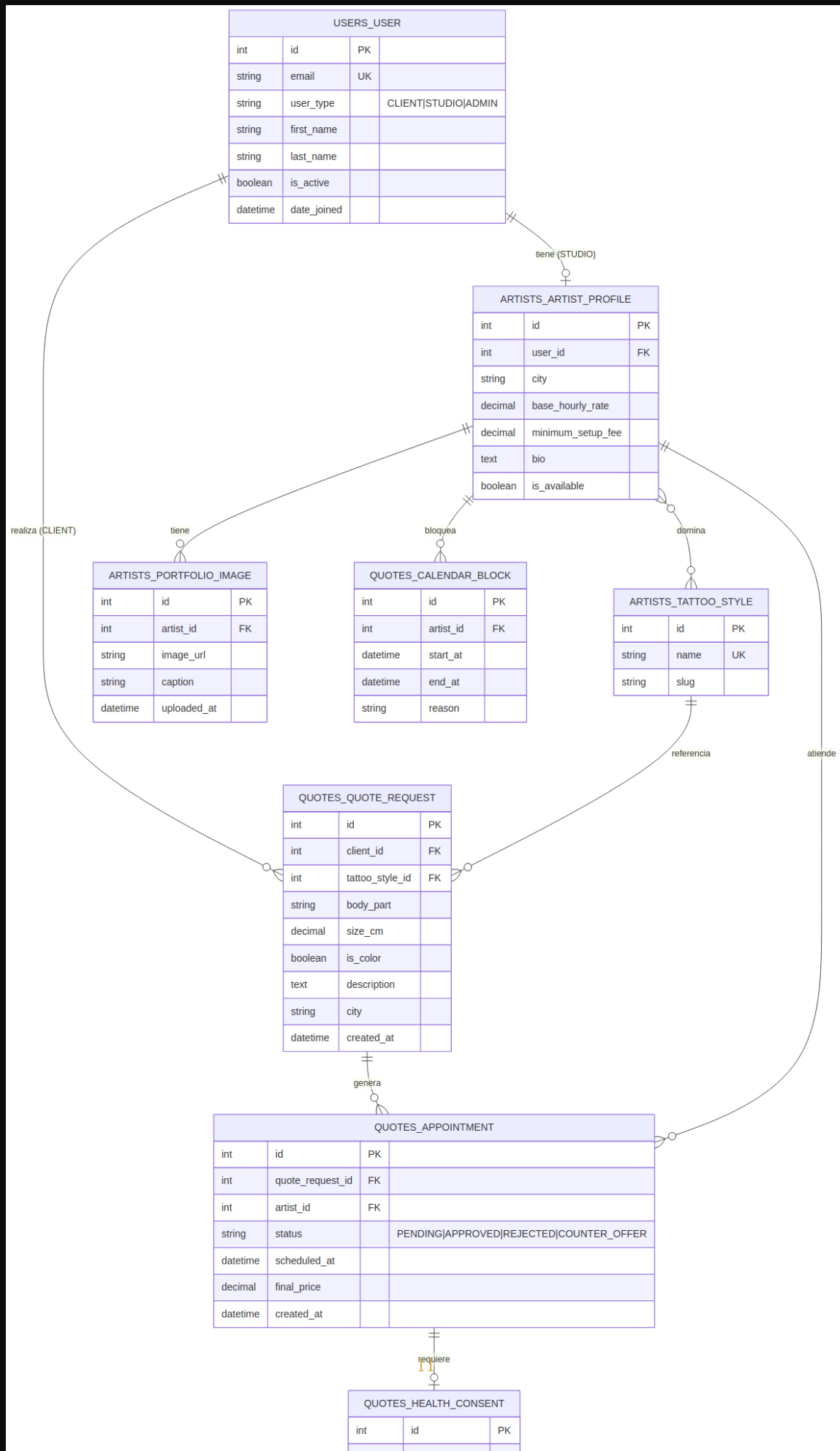
6.1. Descripción

El modelo de datos refleja las entidades del dominio: usuarios con roles, perfiles de artistas, solicitudes de cotización, citas y consentimientos de salud.

6.2. Relaciones principales

- **User (STUDIO) → ArtistProfile:** relación 1:1
- **User (CLIENT) → QuoteRequest:** relación 1:N
- **ArtistProfile ↔ TattooStyle:** M:N (artista domina varios estilos)
- **QuoteRequest → Appointment:** 1:N (un quote puede generar múltiples citas)
- **Appointment → HealthConsent:** 1:1 (opcional hasta que se aprueba la cita)

6.3. Diagrama entidad-relación



7. Normalización de Ciudades

7.1. Descripción

La normalización de ciudades garantiza que el matching artista-cliente funcione con búsquedas exactas. El catálogo `cities-mx.ts` es la fuente de verdad compartida entre frontend y backend.

7.2. Problema que resuelve

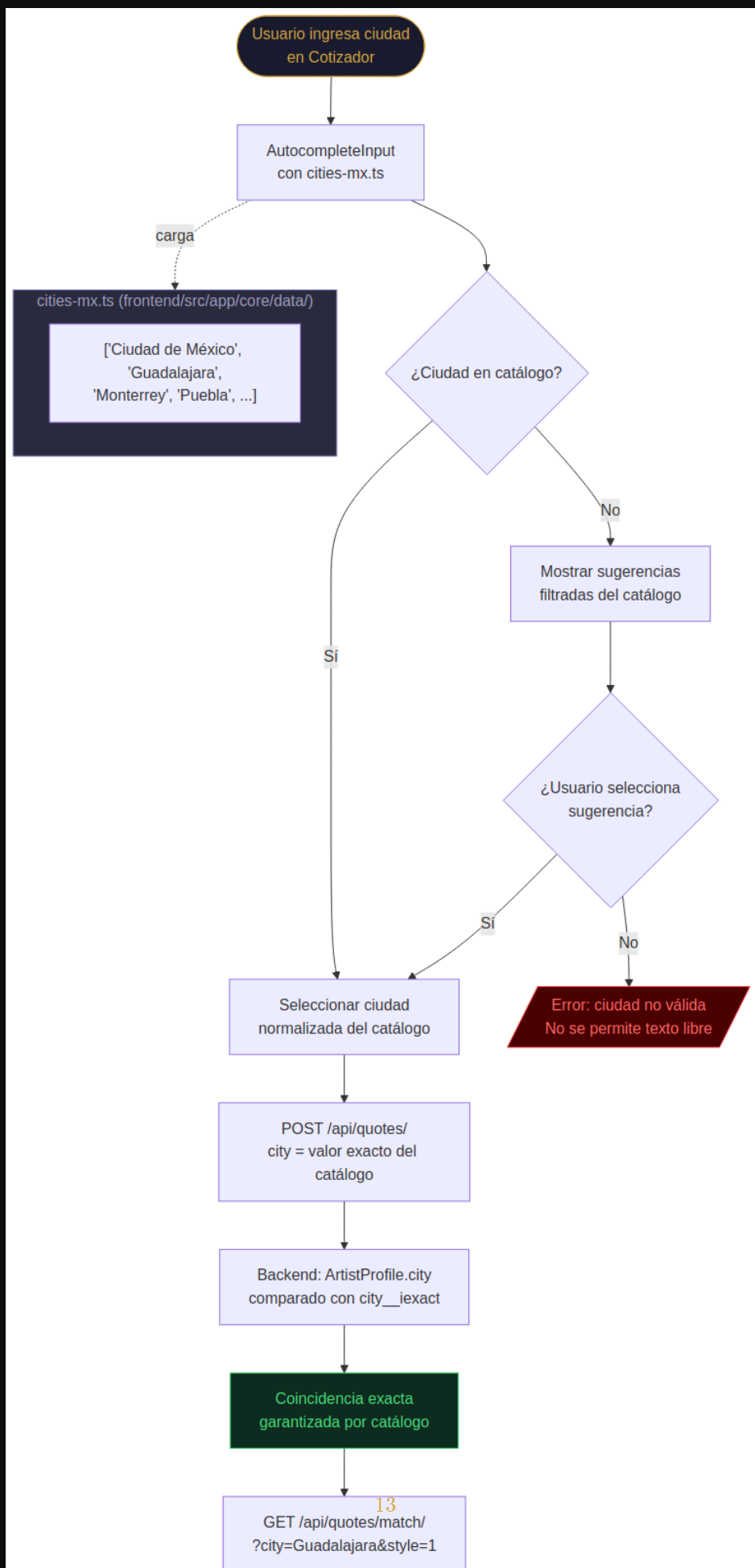
Sin normalización, un cliente que escribe “Gdl” o “guadalajara” no encontraría artistas registrados como “Guadalajara”. El catálogo y el filtro `city__iexact` garantizan coincidencias exactas independientemente de mayúsculas/minúsculas.

7.3. Invariantes del sistema

Reglas de normalización (siempre vigentes)

1. Usar **siempre** el catálogo `frontend/src/app/core/data/cities-mx.ts`
2. **Nunca** texto libre en campos de ciudad
3. Backend filtra con `city__iexact` — el catálogo garantiza la coincidencia
4. **Pendiente (backlog)**: migrar `ArtistProfile.city` a FK hacia tabla normalizada

7.4. Diagrama de flujo



8. Flujo de Contraoferta de Precio y Fecha (BL-95-RF-4)

8.1. Descripción

Cuando un artista no puede aceptar la cita en los términos propuestos por el cliente, puede emitir una **contraoferta** con una nueva fecha y/o nota explicativa. El cliente entonces puede aceptar (transición a APPROVED) pero no rechazar — la única transición válida desde COUNTER_OFFER es APPROVED.

8.2. Validaciones del serializer

AppointmentStatusSerializer — regla de contraoferta

```
class AppointmentStatusSerializer(serializers.Serializer):
    status = ChoiceField(choices=Appointment.Status.choices)
    counter_offer_datetime = DateTimeField(required=False, allow_null=True)
    counter_offer_note = CharField(required=False, default="", allow_blank=True)

    def validate(self, data):
        if data["status"] == "COUNTER_OFFER":
            if not data.get("counter_offer_datetime"):
                raise ValidationError({"counter_offer_datetime": "Requerido."})
```

Estado origen	Estado destino	Actor / Restricción
PENDING	COUNTER_OFFER	Artista dueño de la cita + counter_offer_datetime requerida
PENDING	APPROVED	Artista dueño de la cita
PENDING	REJECTED	Artista dueño de la cita
COUNTER_OFFER	APPROVED	Solo el cliente — única transición válida
COUNTER_OFFER	REJECTED	NO PERMITIDO → 400 Bad Request

Restricción implementada en views.py

Desde COUNTER_OFFER el código retorna 400 si el cliente intenta pasar a REJECTED. Solo APPROVED es válido desde ese estado. Esta restricción es más estricta que la tabla de estados del §4.

8.3. Diagrama de secuencia

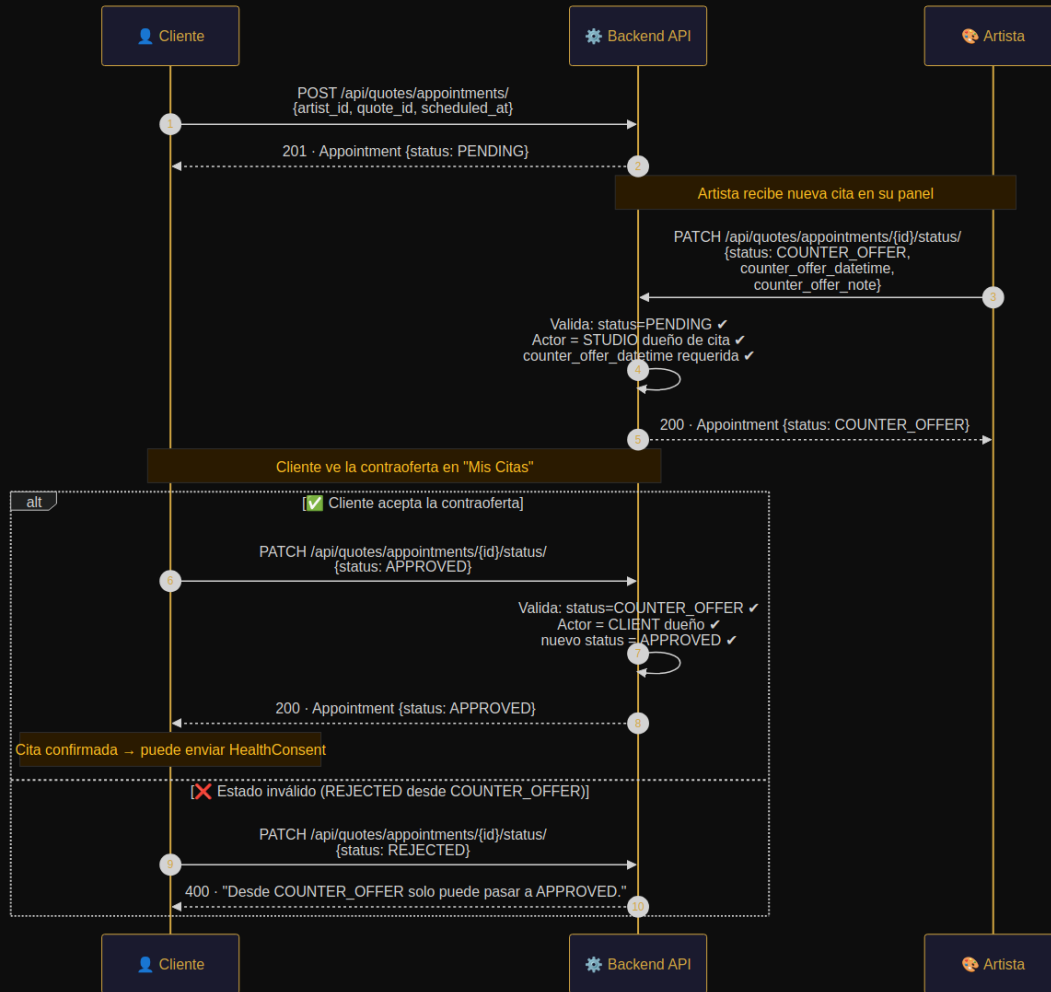


Figure 7: Secuencia de contraoferta: artista propone cambios, cliente acepta o recibe error

9. Validación de Solapamiento en CalendarBlock

9.1. Descripción

El modelo `CalendarBlock` permite al artista bloquear franjas de tiempo. La validación de solapamiento garantiza que no existan dos bloques simultáneos ni que un bloqueo cubra una cita ya confirmada.

9.2. Algoritmo de detección

El solapamiento entre dos rangos (A_{start} , A_{end}) y (B_{start} , B_{end}) existe si y solo si:

$$A_{start} < B_{end} \wedge A_{end} > B_{start}$$

Traducido a ORM de Django:

Query de detección de solapamiento (a agregar en CalendarBlockSerializer)

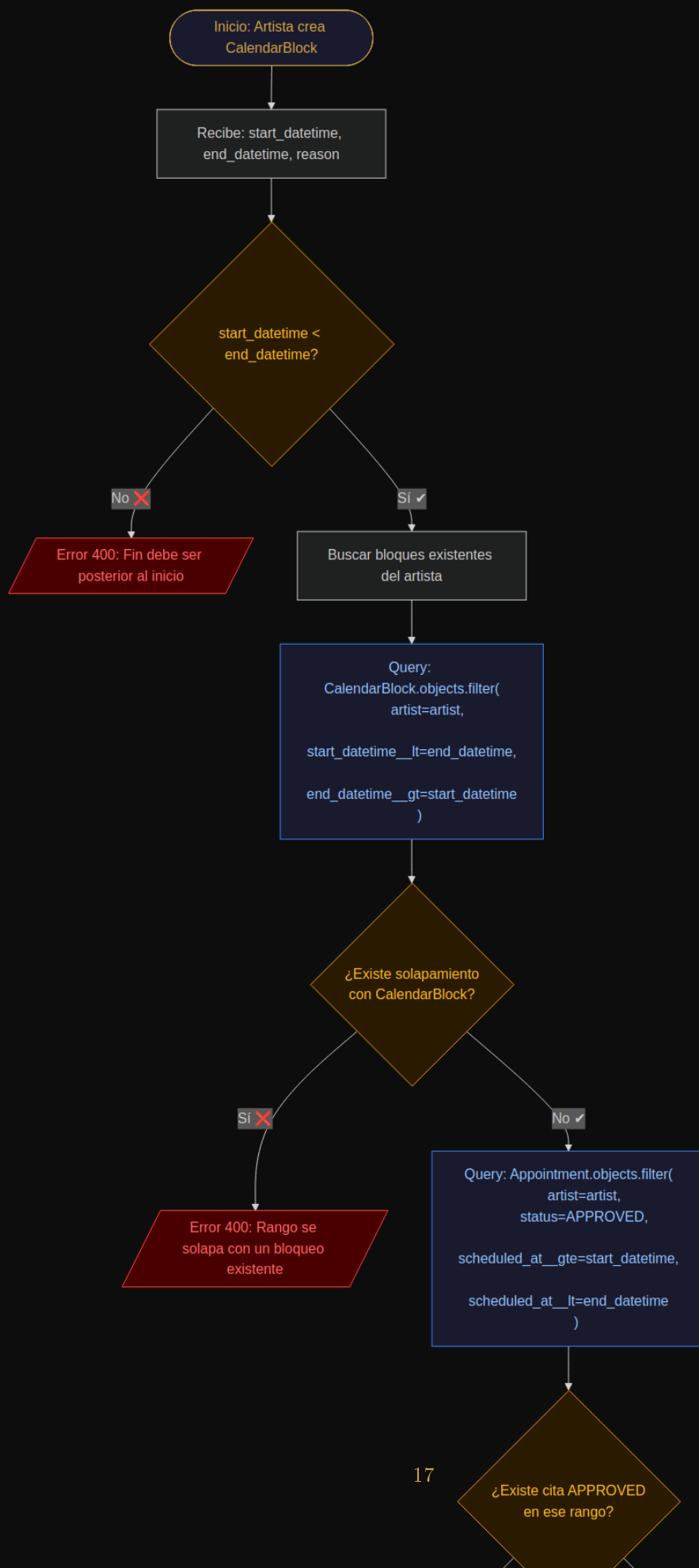
```
# Bloques existentes que se solapan con el nuevo rango
overlapping = CalendarBlock.objects.filter(
    artist=artist,
    start_datetime__lt=end_datetime,
    end_datetime__gt=start_datetime,
)

# Citas APPROVED en ese rango
blocked = Appointment.objects.filter(
    artist=artist,
    status=Appointment.Status.APPROVED,
    scheduled_at__gte=start_datetime,
    scheduled_at__lt=end_datetime,
)
```

9.3. Casos borde

- **Bloques contiguos:** (08:00-10:00) y (10:00-12:00) **no** se solapan porque 10:00 $\nless$ 10:00.
- **Mismo horario exacto:** (08:00-10:00) y (08:00-10:00) **sí** se solapan.
- **Bloqueo parcial:** (09:00-11:00) sobre (08:00-10:00) **sí** se solapa.
- El método `CalendarBlock.full_clean()` ya valida que `start < end` antes de guardar.

9.4. Diagrama de decisión



10. Pipeline OpenAPI → Tipos Angular

10.1. Descripción

El contrato API se sincroniza automáticamente entre backend y frontend mediante el pipeline **drf-spectacular** → **openapi-typescript**. Saltarse este paso produce drift silencioso: los tipos TypeScript quedan desactualizados sin error.

10.2. Cuándo regenerar

Obligatorio regenerar si cambia cualquiera de estos

- Un serializer (campo nuevo, renombrado o eliminado)
- Un ViewSet o APIView (método, endpoint, parámetro)
- Un `@extend_schema` o decorador de drf-spectacular
- Un modelo cuyos campos se exponen directamente en un serializer

10.3. Comandos del pipeline

Paso 1 — Generar schema.yml (backend)

```
cd backend
python manage.py spectacular -file schema.yml
```

Paso 2 — Regenerar tipos TypeScript (frontend)

```
cd $PROJECT_ROOT
npx openapi-typescript backend/schema.yml \
-o frontend/src/app/core/api/types.ts
```

Paso 3 — Detectar drift antes del PR

```
git diff backend/schema.yml
# Si hay cambios, también debe haber diff en frontend/src/app/core/api/types.ts
```

10.4. Diagrama de dependencias

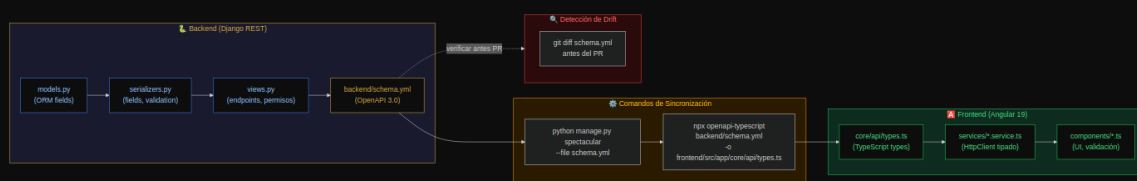


Figure 9: Pipeline de sincronización: `models.py` → `schema.yml` → `types.ts` → componentes Angular

11. Flujo de Cuestionario de Salud (HealthConsent)

11.1. Descripción

El cuestionario de salud es un **requisito legal** bajo la Ley Federal de Protección de Datos Personales en Posesión de Particulares (LFPDPPP, México). Solo puede enviarse una vez por cita y requiere firma digital del cliente.

11.2. Campos del modelo

Campo	Descripción	Obligatorio
has_allergies	¿Tiene alergias? + detalle	No
has_chronic_disease	Diabetes, hipertensión, coagulopatías	No
takes_medication	Anticoagulantes, corticosteroides, etc.	No
is_pregnant	Embarazo o lactancia	No
has_skin_condition	Psoriasis, queloide, dermatitis	No
has_hemophilia	Trastorno de coagulación	No
signature_data	PNG del cliente en base64 (data:image/png;base64,...)	Sí
terms_accepted	Aceptación LFPDPPP	Sí

Reglas de negocio (LFPDPPP)

- **Idempotencia:** el endpoint POST rechaza si ya existe un consent para esa cita (400).
- **Firma obligatoria:** signature_data debe iniciar con "data:image/".
- **Términos obligatorios:** terms_accepted = false → 400.
- **Solo el cliente:** actores STUDIO o ADMIN reciben 403.
- **Retención:** los datos se almacenan indefinidamente por cumplimiento legal.

11.3. Diagrama de secuencia

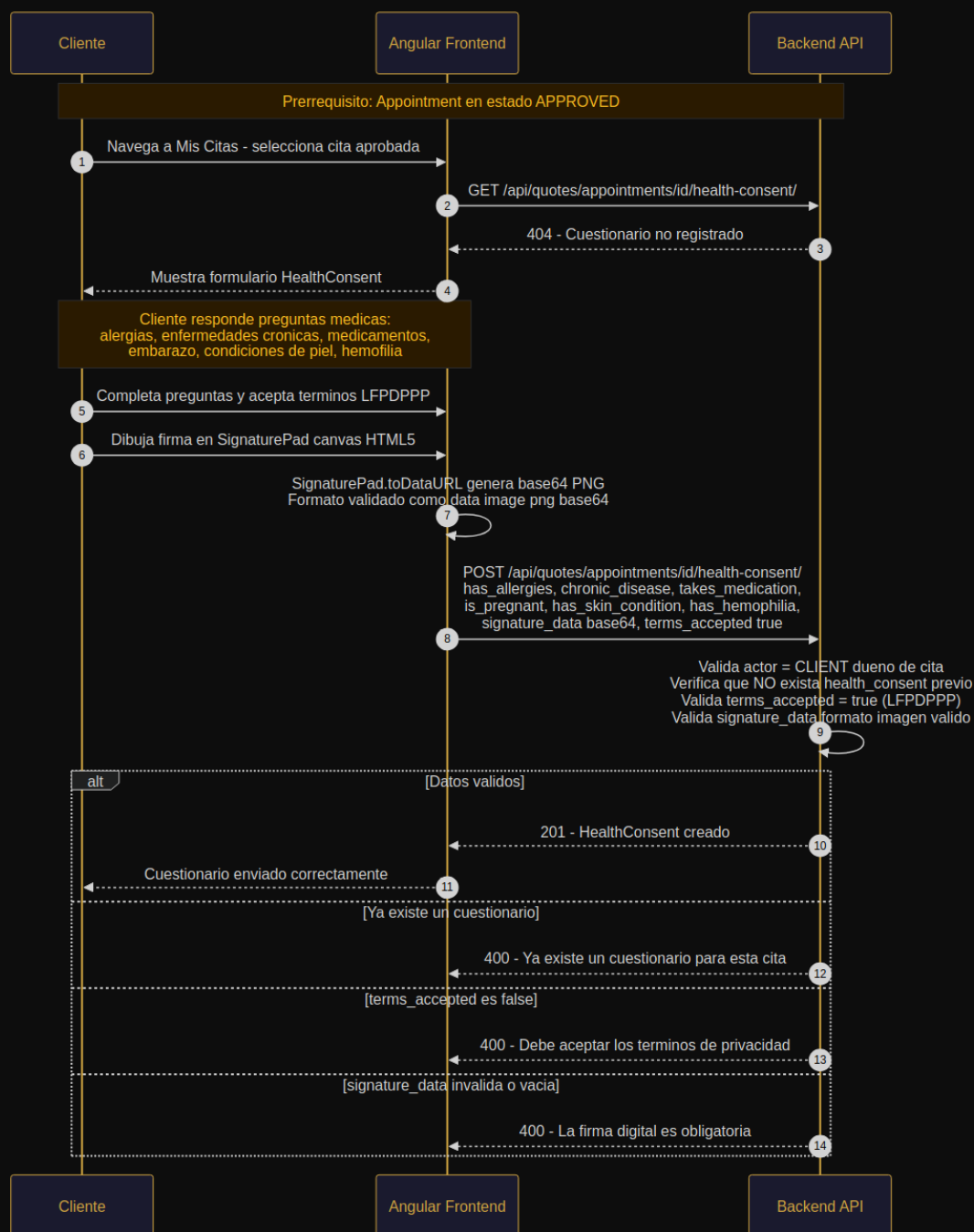


Figure 10: Flujo completo de HealthConsent: firma digital → validaciones LFPDPPP → persistencia

12. Algoritmo de Match (Filtrado de Artistas)

12.1. Descripción

El endpoint `GET /api/quotes/match/` es el corazón del marketplace. Todo el cálculo de precio estimado se ejecuta en **una sola query SQL** mediante `annotate` del ORM de Django, cumpliendo el RNF-2 (≤ 2.5 s de respuesta).

12.2. Parámetros de entrada

Parámetro	Tipo	Obligatorio	Descripción
city	string	Sí	Ciudad del artista (iexact)
style_id	integer	Sí	PK del estilo deseado
size_cm	integer	Sí	Tamaño estimado en cm (≥ 1)
body_part	choice	Sí	Zona del cuerpo
is_color	boolean	Sí	¿A color?
max_price	decimal	No	Presupuesto máximo del cliente

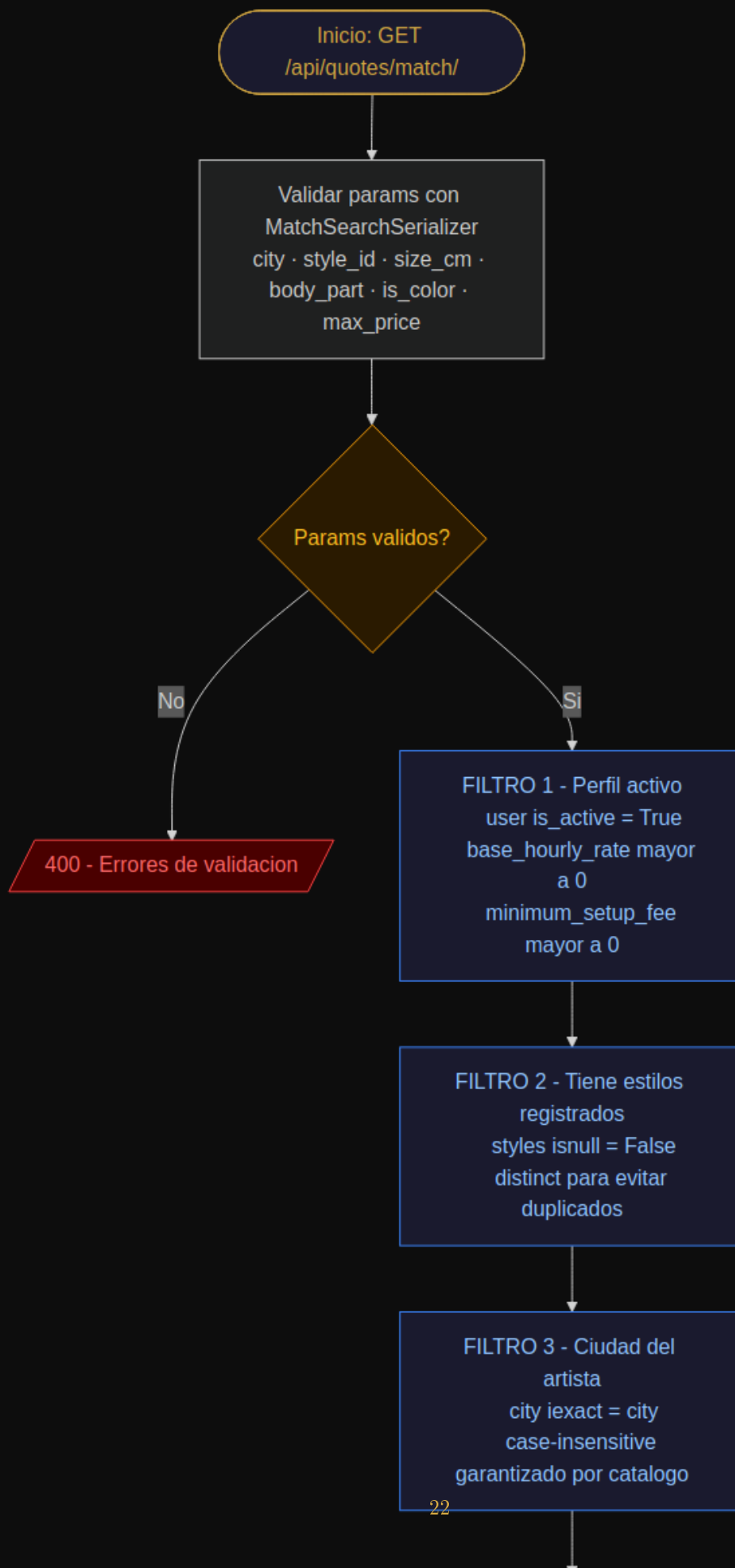
12.3. Pipeline de la query (7 pasos)

SQL compilado por el ORM — ArtistMatchView.get()

```
SELECT id, city, bio, base_hourly_rate, minimum_setup_fee,
GREATEST(
{size_cm} * base_hourly_rate * {zone_mult} * {clr_mult},
minimum_setup_fee
) AS estimated_price,
CASE WHEN styles.id = {style_id} THEN TRUE ELSE FALSE END AS style_match
FROM artists_artistprofile
JOIN users_user ON user_id = users_user.id
WHERE users_user.is_active = TRUE
AND base_hourly_rate > 0
AND minimum_setup_fee > 0
AND city ILIKE '{city}'
ORDER BY style_match DESC, estimated_price ASC
```

El Match actual **no filtra** por CalendarBlock ni por Appointment aprobadas en la fecha deseada. Un artista con agenda llena aparecerá igual en los resultados. Pendiente: agregar parámetro **desired_date** y excluir artistas bloqueados.

12.4. Diagrama de flujo



13. Guards y Permisos por Rol (Angular)

13.1. Descripción

La aplicación implementa tres guards funcionales (`CanActivateFn`) que protegen las rutas según el estado de autenticación y el rol del usuario.

13.2. Guards implementados

authGuard — Verifica sesión activa

```
// Comprueba:
// 1. Existe user en memoria
// 2. Access token en sessionStorage + no expirado (exp × 1000 > Date.now())
// Si falla → redirige a /auth/login?returnUrl={ruta_original}
```

roleGuard(...roles) — Verifica rol específico

```
roleGuard('CLIENT') // Solo clientes
roleGuard('STUDIO') // Solo artistas
// ADMIN pasa siempre (bypass de roles)
// Rol incorrecto → redirige al dashboard del rol real
```

noAuthGuard — Rutas públicas

```
// Si ya hay sesión activa → redirectByRole():
// STUDIO → /studio/dashboard
// CLIENT → /client/dashboard
// ADMIN → /admin/ (Django Admin, fuera de la SPA)
// Evita que usuarios logueados vean landing/login
```

13.3. Árbol de rutas protegidas

Ruta	Guards	Roles permitidos
/ (landing)	noAuthGuard	Sin sesión
/auth/**	—	Público (no guard)
/client/**	authGuard + roleGuard('CLIENT')	CLIENT, ADMIN
/studio/**	authGuard + roleGuard('STUDIO')	STUDIO, ADMIN

13.4. Diagrama de flujo

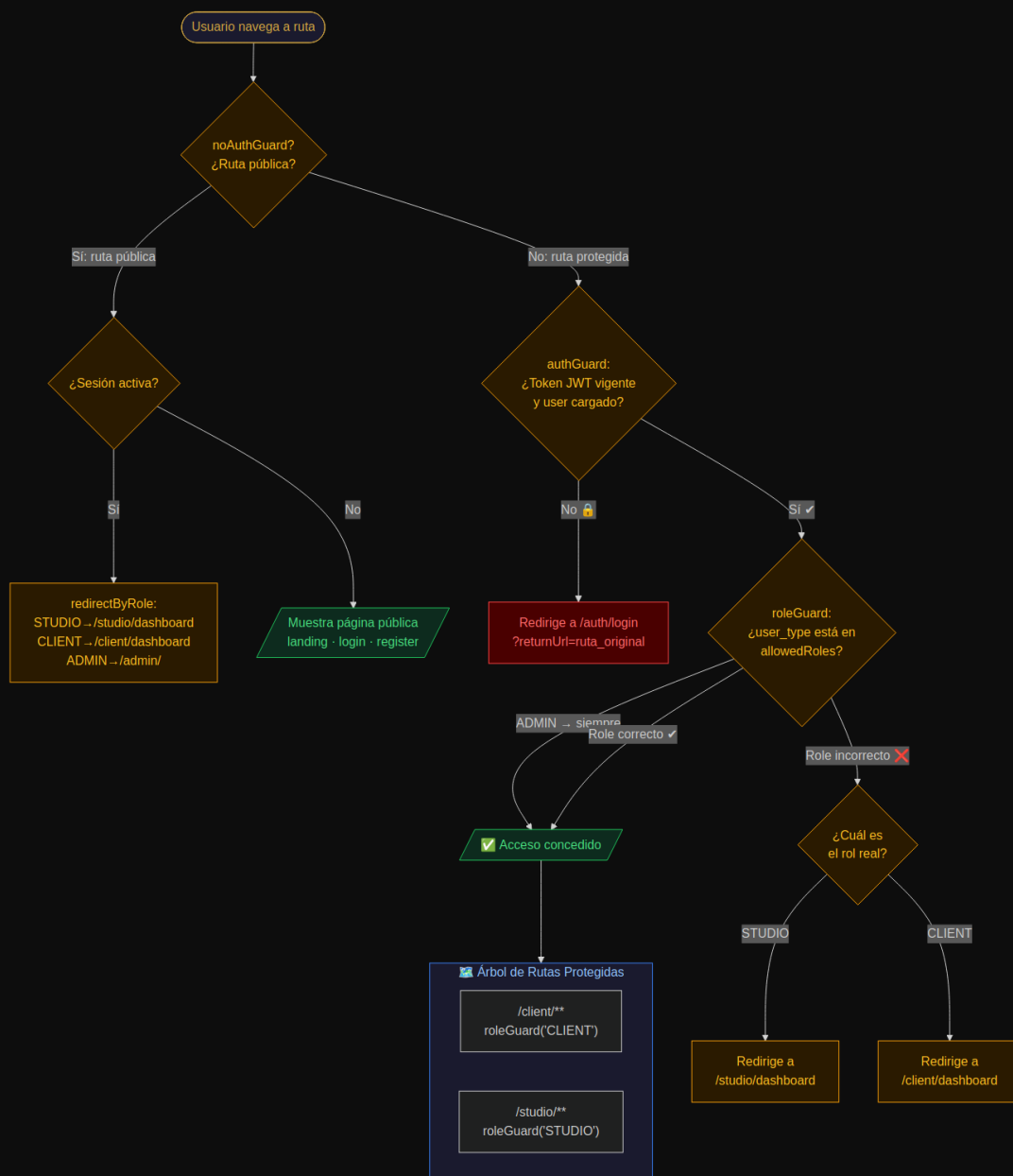


Figure 12: Árbol de decisión de los guards Angular: noAuth → auth → role

14. Permisos DRF en el Backend

14.1. Descripción

Cada endpoint del backend verifica **autenticación + rol + propiedad del recurso**. La función auxiliar `_get_appointment_or_403` centraliza la lógica de ownership.

14.2. Tabla de permisos por endpoint

Endpoint	Método	Quien puede
/api/quotes/	GET	CLIENT (sus quotes)
/api/quotes/	POST	CLIENT
/api/quotes/match/	GET	Público (AllowAnonymous)
/api/quotes/appointments/	GET	CLIENT (las tuyas) · STUDIO (las tuyas) · ADMIN (todas)
/api/quotes/appointments/	POST	Solo CLIENT
/api/quotes/appointments/{id}/status/	PATCH	STUDIO (desde PENDING) · CLIENT (desde COUNTER_OFFER)
/api/quotes/appointments/{id}/health-consent/	POST	Solo CLIENT dueño
/api/quotes/calendar-blocks/	GET	STUDIO (los suyos) · ADMIN (todos)
/api/quotes/calendar-blocks/	POST	Solo STUDIO
/api/quotes/calendar-blocks/{id}/	DELETE	Solo STUDIO dueño

Escalada de privilegios — riesgos y mitigaciones

- Un `STUDIO` intenta cambiar estado de cita de otro artista $\rightarrow$ 403 (verificado con `appt.artist.user != user`).
- Un `CLIENT` intenta enviar `COUNTER_OFFER` $\rightarrow$ 400 (verificado en la máquina de estados).
- Un `STUDIO` intenta crear una cita $\rightarrow$ 403 (`user_type != "CLIENT"`).
- **Pendiente:** custom permission classes (`IsArtistOwner`, `IsClientOwner`) para reemplazar los `if` inline y centralizar la lógica.

14.3. Diagrama de permisos

Figure 13: Árbol de permisos por rol: autenticación $\rightarrow$ rol $\rightarrow$ ownership del recurso

15. Cálculo del Rango de Precio en el Frontend

15.1. Descripción

El frontend calcula un rango de precio **estimado** usando una tarifa de mercado promedio (`BASE_RATE_MXN = 150`) antes de que el cliente elija artista. Este precio es orientativo; el precio definitivo lo calcula el backend con la tarifa real del artista.

15.2. Fórmula frontend

cotizador.component.ts — _calcPrice()

```
const BASE_RATE_MXN = 150; // MXN/cm - tarifa de mercado promedio

function _calcPrice(form): { min: number; max: number } | null {
  const factor = ZONE_MULTIPLIERS[form.body_part];
  const colorMult = form.is_color ? 1.20 : 1.00;
  const price = Math.round(
    BASE_RATE_MXN * form.size_cm * factor * colorMult / 10
  ) * 10;
  return { min: price, max: Math.round(price * 1.35 / 10) * 10 };
}
```

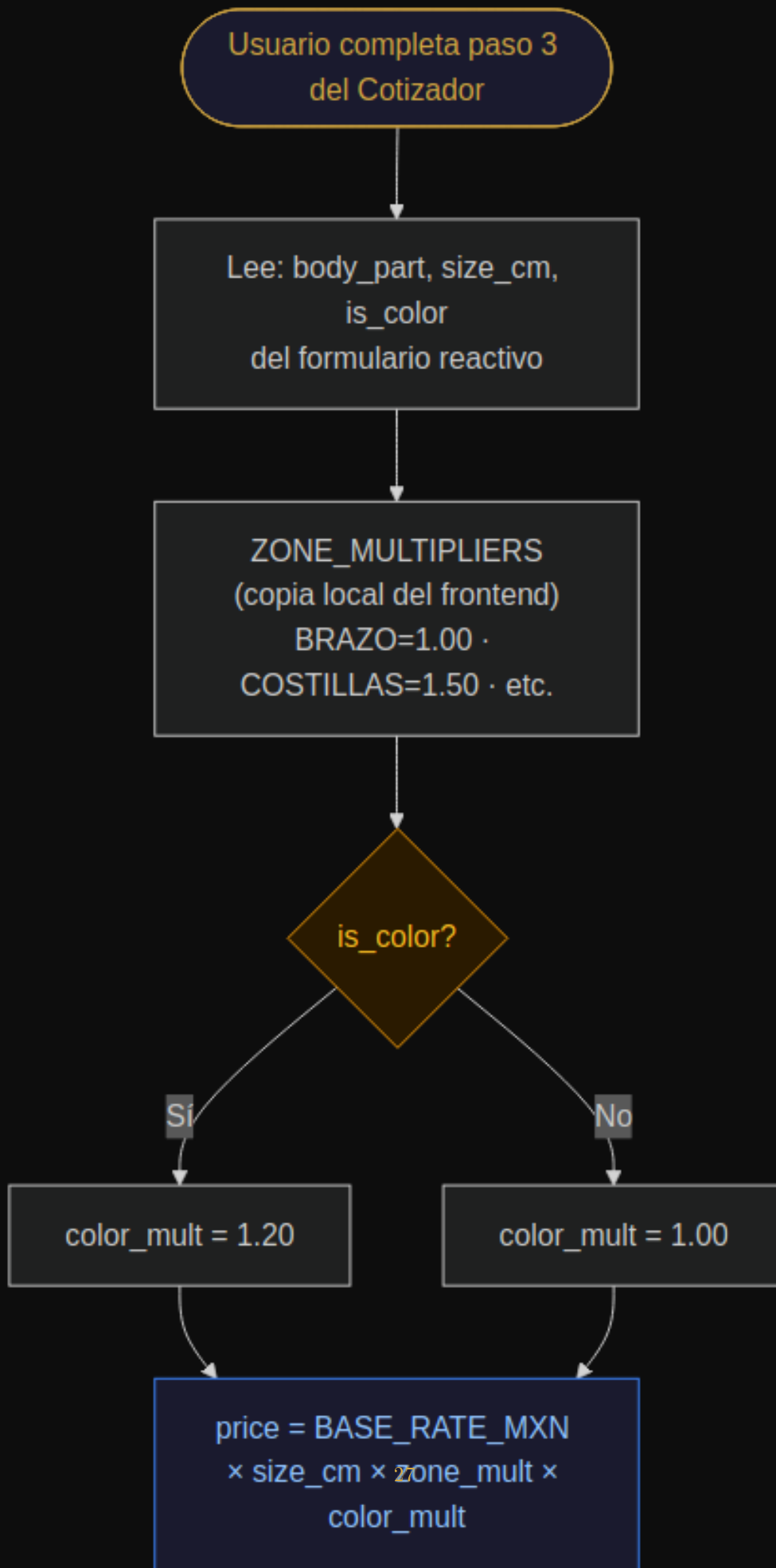
15.3. Diferencia entre precio frontend y precio backend

	Frontend (estimado)	Backend (real)
Tarifa base	BASE_RATE_MXN = 150 (constante)	ArtistProfile.base_hourly_rate (por artista)
Cuándo	Paso 3 del Cotizador (sin artista)	Resultado del Match (con artista)
Propósito	Orientar al cliente	Precio oficial de la tarjeta
Fuente de verdad	No	Sí

Riesgo de desincronización

BASE_RATE_MXN = 150 es una constante arbitraria. Si el mercado sube, el estimado del cotizador podría quedar muy por debajo del precio real del artista, generando una expectativa incorrecta al cliente. Considerar obtenerlo desde un endpoint de configuración o como promedio dinámico.

15.4. Diagrama de flujo



16. Índices de Base de Datos (Performance)

16.1. Descripción

Sin índices adecuados, las queries de Match y listado de citas degradarán con el volumen de datos. Los índices que se describen a continuación están priorizados por impacto estimado.

16.2. Índices recomendados

Modelo	Índice	Prioridad
QuoteRequest	(client_id, created_at DESC)	Alta
Appointment	(artist_id, status, scheduled_at)	Alta
Appointment	(artist_id, scheduled_at) WHERE status='APPROVED'	Alta
CalendarBlock	(artist_id, start_datetime, end_datetime)	Media
ArtistProfile	(city)	Media (afecta Match)

16.3. Implementación con migraciones Django

Agregar en Meta de los modelos

```
class Meta:
    indexes = [
        models.Index(
            fields=["client", "-created_at"],
            name="qr_client_created_idx",
        ),
        models.Index(
            fields=["artist", "status", "scheduled_at"],
            name="appt_artist_status_idx",
        ),
    ]
```

Índice parcial con RunSQL (solo APPROVED)

```
migrations.RunSQL(
    "CREATE INDEX appt_approved_idx ON quotes_appointment
    (artist_id, scheduled_at) WHERE status = 'APPROVED';",
    reverse_sql="DROP INDEX IF EXISTS appt_approved_idx;",
)
```

16.4. Diagrama de índices



Figure 15: Índices recomendados por tabla e impacto en las queries críticas

17. Endpoint GET /api/quotes/cities/ (Backlog UX)

17.1. Descripción

Actualmente el autocomplete de ciudades en el Match usa el catálogo estático `cities-mx.ts` con todas las ciudades de México. Este endpoint retornaría solo las ciudades donde hay artistas activos, mejorando la UX al no mostrar ciudades sin cobertura.

17.2. Implementación propuesta

Query del endpoint (`backend/apps/quotes/views.py`)

```
class ActiveCitiesView(APIView):
    permission_classes = [AllowAny]

    def get(self, request):
        cities = (
            ArtistProfile.objects
            .filter(user__is_active=True, base_hourly_rate__gt=0)
            .values_list("city", flat=True)
            .distinct()
            .order_by("city")
        )
        return Response({"cities": list(cities)})
```

Este endpoint puede cachearse agresivamente porque las ciudades activas cambian raramente:

- **TTL recomendado:** 1 hora (3 600 s)
- **Invalidación:** al activar/desactivar un `ArtistProfile`
- **Implementación:** `@method_decorator(cache_page(3600))` o caché manual con Django-Redis

17.3. Integración con el autocomplete Angular

Intersección de catálogos en el componente Match

```
// 1. Cargar ciudades activas del backend
const activeCities = await this.citiesService.getActiveCities();

// 2. Filtrar el catálogo estático cities-mx.ts
const availableCities = CITIES_MX.filter(c => activeCities.includes(c));

// 3. Solo mostrar ciudades con cobertura en el autocomplete
this.filteredCities = availableCities;
```

17.4. Diagrama de flujo

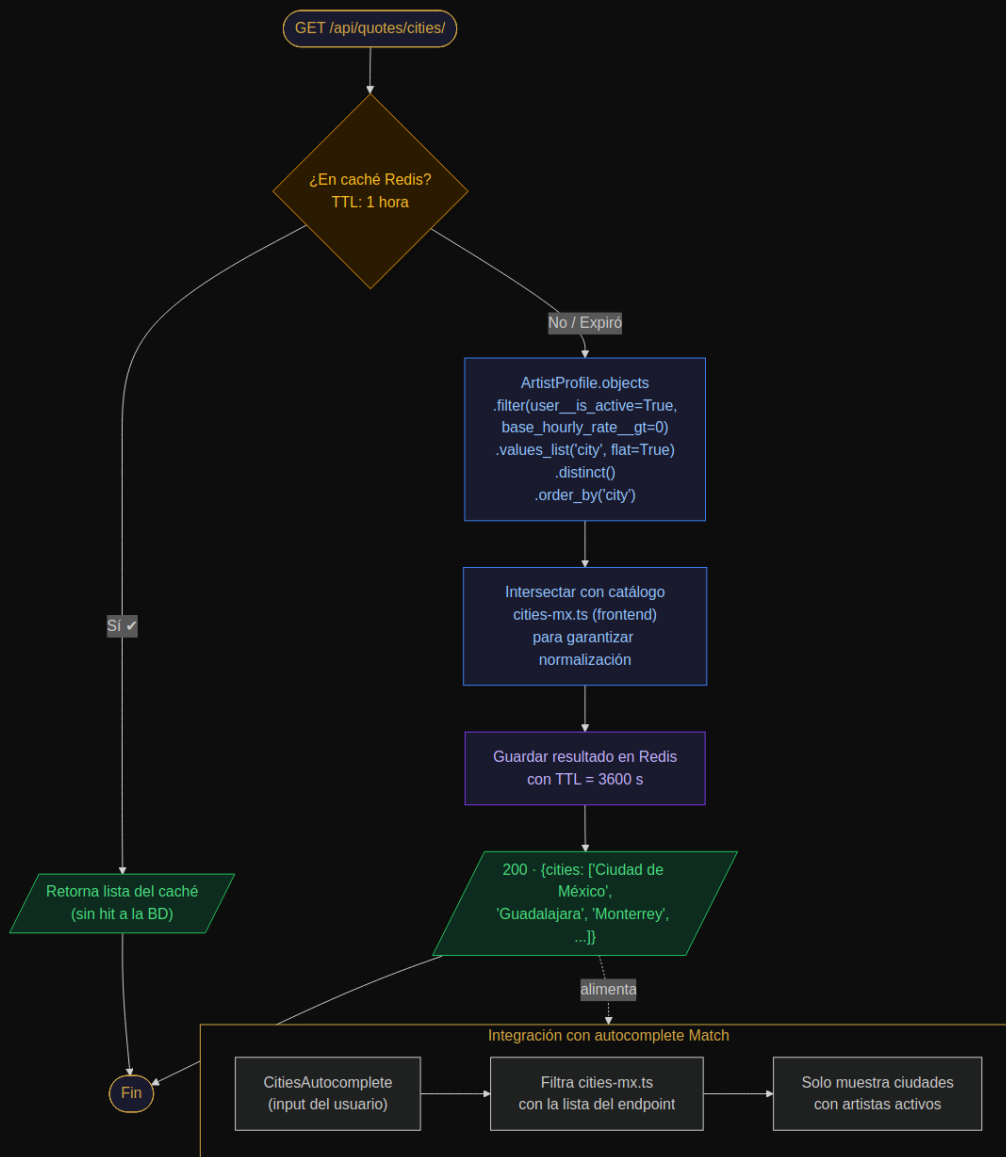


Figure 16: Endpoint de ciudades activas con caché Redis e integración con autocomplete Angular

18. Resumen de Endpoints Críticos

Método	Endpoint	Descripción
POST	/api/auth/token/	Login, retorna access + refresh JWT
POST	/api/auth/token/refresh/	Renueva access token
POST	/api/quotes/	Crea QuoteRequest (4 pasos cotizador)
GET	/api/quotes/match/	Artistas disponibles por ciudad+estilo
POST	/api/quotes/appointments/	Agenda cita (PENDING)
PATCH	/api/quotes/appointments/{id}/status/	Cambia estado (artista)
POST	/api/quotes/appointments/{id}/health-consent/	Guarda consentimiento
GET	/api/artists/styles/	Catálogo de estilos de tatuaje
GET	/api/quotes/calendar-blocks/	Bloques de calendario del artista
POST	/api/quotes/calendar-blocks/	Artista bloquea franja de tiempo
DELETE	/api/quotes/calendar-blocks/{id}/	Elimina un bloqueo
GET	/api/quotes/cities/	(Backlog) Ciudades con artistas activos

Estado de la documentación

Algoritmo / Flujo	Sección	Estado
Precio canónico (fórmula + multiplicadores)	§2	
Flujo Cotizador → Match → Cita	§3	
Máquina de estados de Appointment	§4	
Gestión de sesión JWT + renovación automática	§5	
ERD de base de datos	§6	
Normalización de ciudades	§7	
Flujo de Contraoferta (BL-95-RF-4)	§8	
Validación de solapamiento CalendarBlock	§9	
Pipeline OpenAPI → Tipos Angular	§10	
Flujo de HealthConsent (LFPDPPP)	§11	
Algoritmo de Match (7 pasos, SQL)	§12	
Guards y permisos por rol (Angular)	§13	
Permisos DRF en el backend	§14	
Cálculo del rango de precio en frontend	§15	
Índices de base de datos (performance)	§16	
Endpoint GET /api/quotes/cities/ (backlog)	§17	

BlackLine Engineering · Documentación técnica completa — v2.0.0
claude-sonnet-4-6 · Mayo 2026